Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-214БВ-24

Студент: ГеннихА.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата:13.10.25

Москва, 2025

Постановка задачи

Вариант 7.

Разработать программу на языке Си, состоящую из двух процессов (родительского и дочернего), которые взаимодействуют через разделяемую память (shared memory) с синхронизацией через семафоры.

Общий метод и алгоритм решения

Использованные системные вызовы:

shm_open() - создает или открывает именованный объект разделяемой памяти (файловый дескриптор для области в RAM)

ftruncate() - устанавливает размер объекта разделяемой памяти (выделяет физическую память под объект)

mmap() - отображает объект разделяемой памяти в адресное пространство процесса (возвращает указатель, по которому можно обращаться к памяти)

munmap() - удаляет отображение разделяемой памяти из адресного пространства процесса

shm_unlink() - удаляет именованный объект разделяемой памяти из системы (освобождает ресурсы)

close() - закрывает файловый дескриптор объекта разделяемой памяти

sem_open() - создает или открывает именованный семафор (счетчик для синхронизации доступа)

sem_wait() - уменьшает счетчик семафора на 1; если счетчик равен 0, процесс блокируется до освобождения ресурса

sem_post() - увеличивает счетчик семафора на 1; если есть процессы, ожидающие на sem_wait, один из них просыпается

sem_close() - закрывает семафор в текущем процессе (освобождает дескриптор)

sem_unlink() - удаляет именованный семафор из системы

void cleanup() - Последовательно освобождает: Отключение shared memory (munmap) Закрытие файлового дескриптора (close) Удаление объекта shared memory (shm_unlink) Удаление семафоров (sem_unlink)

error_exit() - Обрабатывает ошибки и завершает программу

В работе используется подход межпроцессного взаимодействия через именованную разделяемую память (POSIX shared memory) с синхронизацией доступа посредством именованных семафоров. Родительский процесс создает все необходимые объекты IPC с уникальными именами на основе своего PID, что позволяет изолировать разные экземпляры программы друг от друга. После создания разделяемой памяти (shm_open, ftruncate, mmap) и двух семафоров (один для синхронизации записи, другой для чтения), родитель порождает дочерний процесс через fork. В дочернем процессе стандартный ввод перенаправляется на файл с числами (freopen), после чего образ процесса заменяется программой child (execl), которой передаются имена созданных объектов. Дочерний процесс подключается к существующей разделяемой памяти и семафорам, читает числа из перенаправленного stdin, вычисляет их сумму и через механизм семафоров (sem_wait/sem_post) синхронизированно записывает результат в разделяемую память. Родитель, ожидая на семафоре, получает сигнал о готовности данных, читает результат из общей памяти, выводит его на экран и выполняет полную очистку всех созданных объектов (munmap, shm_unlink, sem_unlink). После парсинга аргументов командной строки программа генерирует случайный набор точек и

инициализирует первые K точек как начальные центры. Далее запускается итеративный процесс: на каждом шаге массив точек делится на равные части по числу потоков, создаётся заданное количество потоков через системный вызов pthread_create, каждый поток обрабатывает свой диапазон точек, определяя для каждой ближайший центр и при необходимости поднимая атомарный флаг changed. После завершения всех потоков (ожидание через pthread_join) главный поток пересчитывает центры функцией recalculate_centers и проверяет флаг — если изменений не было, алгоритм завершается.

Код программы

Parent.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <semaphore.h>
#include <signal.h>
#include <errno.h>

#define SHM_SIZE 4096

typedef struct {
    float result;
    int data_ready;
    int child_done;
} shared_data_t;

void error_exit(const char *text) {
    perror(text);
    exit(EXIT_FAILURE);
}

void cleanup(const char *shm_name, const char *sem_parent_name,
             const char *sem_child_name, void *shm_ptr, int shm_fd) {
    if (shm_ptr != NULL && shm_ptr != MAP_FAILED) {
        munmap(shm_ptr, SHM_SIZE);
    }
    if (shm_fd >= 0) {
        close(shm_fd);
    }
    if (shm_name != NULL) {
        shm_unlink(shm_name);
    }
    if (sem_parent_name != NULL) {
        sem_unlink(sem_parent_name);
    }
    if (sem_child_name != NULL) {
```

```
        sem_unlink(sem_child_name);
    }
}

int main() {
    pid_t pid;
    char filename[256];
    char shm_name[256];
    char sem_parent_name[256];
    char sem_child_name[256];

    sprintf(shm_name, "/shm_%d", getpid());
    sprintf(sem_parent_name, "/sem_parent_%d", getpid());
    sprintf(sem_child_name, "/sem_child_%d", getpid());

    printf("Родительский процесс PID: %d\n", getpid());
    printf("Имя shared memory: %s\n", shm_name);
    printf("Имя семафора родителя: %s\n", sem_parent_name);
    printf("Имя семафора ребенка: %s\n", sem_child_name);

    printf("Введите имя файла: ");
    if (fgets(filename, sizeof(filename), stdin) == NULL) {
        error_exit("fgets");
    }

    filename[strcspn(filename, "\n")] = '\0';

    if (strlen(filename) == 0) {
        fprintf(stderr, "Введите имя файла\n");
        exit(EXIT_FAILURE);
    }

    int shm_fd = shm_open(shm_name, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {
        error_exit("shm_open");
    }

    if (ftruncate(shm_fd, SHM_SIZE) == -1) {
        cleanup(shm_name, NULL, NULL, NULL, shm_fd);
        error_exit("Уменьшите файл");
    }

    void *shm_ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
```

```
                    MAP_SHARED, shm_fd, 0);
    if (shm_ptr == MAP_FAILED) {
        cleanup(shm_name, NULL, NULL, NULL, shm_fd);
        error_exit(MAP_FAILED);
    }

    shared_data_t *shared_data = (shared_data_t *)shm_ptr;
    shared_data->data_ready = 0;
    shared_data->child_done = 0;

    sem_t *sem_parent = sem_open(sem_parent_name, O_CREAT, 0666, 0);
    if (sem_parent == SEM_FAILED) {
        cleanup(shm_name, NULL, NULL, shm_ptr, shm_fd);
        error_exit("sem_open parent");
    }

    sem_t *sem_child = sem_open(sem_child_name, O_CREAT, 0666, 1);
    if (sem_child == SEM_FAILED) {
        sem_unlink(sem_parent_name);
        cleanup(shm_name, sem_parent_name, NULL, shm_ptr, shm_fd);
        error_exit("sem_open child");
    }

    sem_close(sem_parent);
    sem_close(sem_child);

    pid = fork();
    if (pid == -1) {
        cleanup(shm_name, sem_parent_name, sem_child_name, shm_ptr, shm_fd);
        error_exit("fork");
    }

    if (pid == 0) {
        FILE *file = freopen(filename, "r", stdin);
        if (file == NULL) {
            fprintf(stderr, "Не удалось открыть файл ребенка %s\n", filename);
            exit(EXIT_FAILURE);
        }

        execl("./child", "child", shm_name, sem_parent_name, sem_child_name,
NULL);

        perror("execl");
```

```
        exit(EXIT_FAILURE);
    } else {
        printf("Дочерний процесс создан (PID: %d)\n", pid);

        sem_parent = sem_open(sem_parent_name, 0);
        if (sem_parent == SEM_FAILED) {
            cleanup(shm_name, sem_parent_name, sem_child_name, shm_ptr,
shm_fd);
            error_exit("sem_open parent in parent process");
        }

        if (sem_wait(sem_parent) == -1) {
            sem_close(sem_parent);
            cleanup(shm_name, sem_parent_name, sem_child_name, shm_ptr,
shm_fd);
            error_exit("sem_wait parent");
        }

        if (shared_data->data_ready) {
            printf("Получен результат: %.2f\n", shared_data->result);
        }

        sem_post(sem_child);

        wait(NULL);

        sem_close(sem_parent);

        cleanup(shm_name, sem_parent_name, sem_child_name, shm_ptr, shm_fd);
    }

    return 0;
}
```

Child.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <semaphore.h>
```

```
#include <errno.h>

#define SHM_SIZE 4096

typedef struct {
    float result;
    int data_ready;
    int child_done;
} shared_data_t;

void error_exit(const char *msg) {
    perror(msg);
    exit(EXIT_FAILURE);
}

int main(int argc, char *argv[]) {
    if (argc != 4) {
        fprintf(stderr, "Использование: %s <shm_name> <sem_parent_name>
<sem_child_name>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    char *shm_name = argv[1];
    char *sem_parent_name = argv[2];
    char *sem_child_name = argv[3];

    printf("Дочерний процесс PID: %d\n", getpid());
    printf("Shared memory: %s\n", shm_name);
    printf("Семафор родителя: %s\n", sem_parent_name);
    printf("Семафор ребенка: %s\n", sem_child_name);

    int shm_fd = shm_open(shm_name, O_RDWR, 0666);
    if (shm_fd == -1) {
        error_exit("child: shm_open");
    }

    void *shm_ptr = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                         MAP_SHARED, shm_fd, 0);
    if (shm_ptr == MAP_FAILED) {
        close(shm_fd);
        error_exit("child: mmap");
    }
```

```
shared_data_t *shared_data = (shared_data_t *)shm_ptr;

sem_t *sem_parent = sem_open(sem_parent_name, 0);
if (sem_parent == SEM_FAILED) {
    munmap(shm_ptr, SHM_SIZE);
    close(shm_fd);
    error_exit("child: sem_open parent");
}

sem_t *sem_child = sem_open(sem_child_name, 0);
if (sem_child == SEM_FAILED) {
    sem_close(sem_parent);
    munmap(shm_ptr, SHM_SIZE);
    close(shm_fd);
    error_exit("child: sem_open child");
}


float sum = 0.0;
float num;
int count = 0;
char line[256];

while (fgets(line, sizeof(line), stdin) != NULL) {
    char *token = strtok(line, " \t\n");
    while (token != NULL) {
        char *endptr;
        num = strtof(token, &endptr);
        if (*endptr == '\0') {
            sum += num;
            count++;
            printf("Прочитано число %.2f, сумма = %.2f\n", num, sum);
        }
        token = strtok(NULL, " \t\n");
    }
}

printf("Всего чисел: %d, сумма = %.2f\n", count, sum);

if (sem_wait(sem_child) == -1) {
    perror("child: sem_wait child");
} else {
    shared_data->result = sum;
```

```
        shared_data->data_ready = 1;

        printf("Результат записан в shared memory\n");

        if (sem_post(sem_parent) == -1) {
            perror("child: sem_post parent");
        }
    }

    sem_close(sem_parent);
    sem_close(sem_child);
    munmap(shm_ptr, SHM_SIZE);
    close(shm_fd);

    printf("Дочерниый процесс завершил работу\n");

    return 0;
}
```

Протокол работы программы

Тестирование:

Data.txt:

10.5 20.3 30.7
5.2 15.8
25.1

```
hoodrich@DESKTOP-DUQ2OCS:~/OSI_LAB/laba_3$ ./parent
Родительский процесс PID: 8289
Имя shared memory: /shm_8289
Имя семафора родителя: /sem_parent_8289
Имя семафора ребенка: /sem_child_8289
Введите имя файла: data.txt
Дочерний процесс создан (PID: 8320)
Дочерний процесс PID: 8320
Shared memory: /shm_8289
Семафор родителя: /sem_parent_8289
Семафор ребенка: /sem_child_8289
Прочитано число 10.50, сумма = 10.50
Прочитано число 20.30, сумма = 30.80
Прочитано число 30.70, сумма = 61.50
Прочитано число 5.20, сумма = 66.70
Прочитано число 15.80, сумма = 82.50
Прочитано число 25.10, сумма = 107.60
Всего чисел: 6, сумма = 107.60
Результат записан в shared memory
Дочерниый процесс завершил работу
Родитель: получен результат: 107.60
```

Strace:

```
hoodrich@DESKTOP-DUQ2OCS:~/OSI_LAB/laba_3$ strace ./parent
execve("./parent", ["./parent"], 0x7ffe11a9ce30 /* 36 vars */) = 0
brk(NULL)                               = 0x559e58a60000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f3dec871000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20203, ...}) = 0
mmap(NULL, 20203, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f3dec86c000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f3dec65a000
mmap(0x7f3dec682000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f3dec682000
mmap(0x7f3dec80a000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1b0000) = 0x7f3dec80a000
mmap(0x7f3dec859000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x7f3dec859000
mmap(0x7f3dec85f000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f3dec85f000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f3dec657000
arch_prctl(ARCH_SET_FS, 0x7f3dec657740) = 0
set_tid_address(0x7f3dec657a10)         = 8693
set_robust_list(0x7f3dec657a20, 24)     = 0
rseq(0x7f3dec658060, 0x20, 0, 0x53053053) = 0
mprotect(0x7f3dec859000, 16384, PROT_READ) = 0
mprotect(0x559e350c6000, 4096, PROT_READ) = 0
mprotect(0x7f3dec8a9000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x7f3dec86c000, 20203)           = 0
getpid()                                = 8693
getpid()                                = 8693
getpid()                                = 8693
getpid()                                = 8693
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x5), ...}) = 0
getrandom("\x50\xb4\xfd\xf7\x29\x35\x87\x15", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x559e58a60000
brk(0x559e58a81000)                     = 0x559e58a81000
write(1,
"\320\240\320\276\320\264\320\270\321\202\320\265\320\273\321\214\321\201\320\272\320\270\320\271
\320\277\321\200\320\276\321"..., 50Родительский процесс PID: 8693
) = 50
write(1, "\320\230\320\274\321\217 shared memory: /shm_8693\n", 32Имя shared memory: /shm_8693
) = 32
write(1, "\320\230\320\274\321\217 \321\201\320\265\320\274\320\260\321\204\320\276\321\200\320\260
\321\200\320\276\320\264\320\270"..., 59Имя семафора родителя: /sem_parent_8693
) = 59
write(1, "\320\230\320\274\321\217 \321\201\320\265\320\274\320\260\321\204\320\276\321\200\320\260
```

```
\321\200\320\265\320\261\320\265"..., 56Имя семафора ребенка: /sem_child_8693
) = 56
fstat(0, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x5), ...}) = 0
write(1, "\320\222\320\262\320\265\320\264\320\270\321\202\320\265 \320\270\320\274\321\217
\321\204\320\260\320\271\320\273\320\260"..., 34Введите имя файла: ) = 34
```

Вывод

В ходе выполнения работы были приобретены навыки управления процессами в Linux и организации межпроцессного взаимодействия через разделяемую память (shm_open, mmap) с синхронизацией семафорами (sem_open, sem_wait, sem_post). Разработана программа из двух процессов: родитель создает уникальные объекты, запускает дочерний процесс через fork/execl, ребенок читает числа из файла, вычисляет сумму и через семафоры передает результат в shared memory родителю. Все системные вызовы обрабатываются с проверкой ошибок, ресурсы гарантированно освобождаются (munmap, shm_unlink, sem_unlink).